



Aggregatore della programmazione dei cinema dell'Umbria

Team RepCode

Emanuele Ceccariglia · Elio Casciola · Andrea Cestelli · Yonas Burka

Ingegneria del Software — ITS Umbria Academy, a.a. 2025/2026 · Prof. Montecchiani

Esposizione — 14 luglio 2026

Il problema

Com'è oggi

Per sapere cosa danno stasera a Perugia devi aprire tre siti diversi, ognuno fatto a modo suo.

PostModernissimo, The Space Corciano e UCI Perugia: programmazioni sparse, formati diversi, nessun colpo d'occhio.

La nostra idea

Una sola app che raccoglie i film, gli orari e i cinema dai tre siti e li mette in un posto solo.

I dati non li scriviamo a mano: li raccoglie in automatico uno scraper.

Gratuita, senza registrazione, sia su web sia su smartphone.

Cosa fa, dal punto di vista di chi la usa

- La home mostra il cartellone del giorno, con una barra per scegliere la data fra i prossimi sette giorni.
- Il dettaglio di un film raccoglie gli orari cinema per cinema, con il link diretto per comprare il biglietto.
- La ricerca trova un film per titolo mentre scrivi.
- La mappa mostra i tre cinema con i loro indirizzi; toccandone uno si apre in Google Maps.
- È la stessa applicazione, dallo stesso codice, sia nel browser sia su iPhone e Android.

Come è fatto: una catena in tre stadi



Ogni stadio ha un compito solo e passa il testimone al successivo con un contratto ben preciso: file JSON fra scraper e backend, chiamate REST fra backend e app.

Il vantaggio: ogni pezzo si può sviluppare e collaudare da solo. Lo scraper lavora di notte, il backend carica i dati nel database, l'app li mostra.

Che cosa deve fare il sistema

ID	Requisito	Priorità
RF-01	Mostrare la lista dei cinema dell'Umbria con nome, città e sito	Alta
RF-02	Mostrare la programmazione di ogni cinema: film, orari e giorni	Alta
RF-03	Mostrare i cinema su una mappa, con marcatori che si possono toccare	Alta
RF-04	Filtrare la programmazione per data (oggi, domani, questa settimana)	Alta
RF-05	Filtrare i cinema per comune o per zona	Media
RF-06	Mostrare la scheda di un film: genere, durata, locandina, trama	Media
RF-07	Dare il link al sito del cinema per comprare il biglietto	Alta
RF-08	Aggiornare i dati da solo, almeno una volta al giorno	Alta
RF-09	Raccogliere i dati rispettando le regole di ogni sito (robots.txt)	Alta
RF-10	Cercare un film per titolo	Media
RF-11	Avvisare se i dati di un cinema sono vecchi di più di 48 ore	Bassa

Come deve comportarsi, e cosa viene prima

Requisiti non funzionali

- Funziona su iPhone e Android dallo stesso codice
- I dati restano aggiornati entro 24 ore dalla fonte
- Nessun account e nessun tracciamento dell'utente
- Rispetta le regole dei siti: al massimo una richiesta al secondo, di notte
- Le schermate principali si aprono in meno di due secondi
- Il backend è raggiungibile almeno il 95% del tempo
- Si può estendere ad altre regioni senza riscrivere il codice

Cosa entra nell'MVP, e cosa no

Ogni requisito è stato pesato per importanza e difficoltà.

Fatto subito. lista, programmazione, mappa, filtro data, scheda film, link acquisto

Curato con attenzione. lo scraping automatico, la parte più difficile

Aggiunto se c'era tempo. filtro per zona e ricerca per titolo

Rimandato. l'avviso sui dati vecchi (poco importante)

Come abbiamo lavorato: uno sprint per pezzo

Sprint 1

Scraper

fatto

Sprint 2

Backend e API

fatto

Sprint 3

App collegata ai dati

fatto

Sprint 4

Mappa e dettaglio

fatto

Sprint 5

Deploy e avviso dati

in parte

L'ordine non è casuale: segue le dipendenze fra i pezzi.

Prima lo scraper, per capire subito se i siti erano davvero leggibili. Poi il backend, che dà all'app dati veri su cui lavorare. Infine l'app. Ogni sprint chiude qualcosa che si può mostrare.

Le versioni sono uscite un po' alla volta: prima le sole API, poi l'app completa (è la versione di oggi), e una versione futura con l'avviso sui dati vecchi e la pubblicazione online.

Scraper: dai siti dei cinema ai dati puliti

Ogni cinema ha il suo modulo di raccolta: stessa interfaccia, tecnica diversa a seconda di com'è fatto il sito.

● PostModernissimo

Sito in Next.js: leggiamo i dati della pagina e l'HTML

● The Space

Ha un'API con login; un browser automatico fa da riserva

● UCI

API interna non documentata, ricostruita osservando la rete

Poi, sui film raccolti, la stessa catena di passaggi:

Pulisce
i titoli



Toglie i
duplicati



Aggiunge dati
da Wikidata



Segna cosa
è cambiato



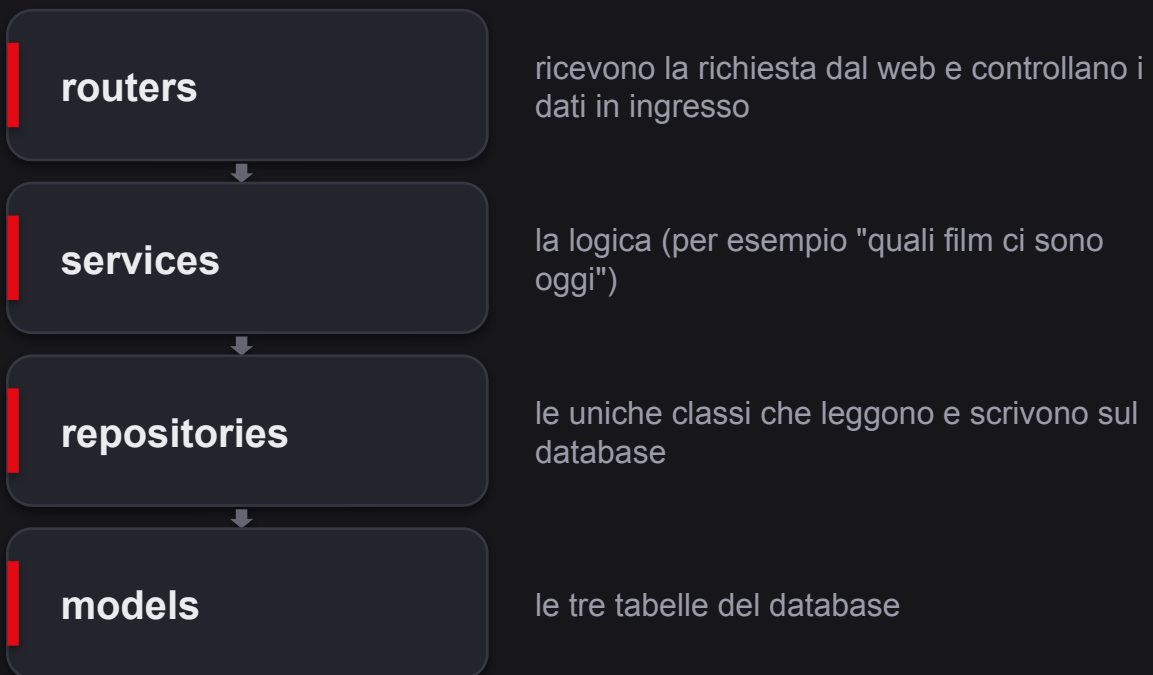
Salva
tre file JSON

Pulire i titoli serve a capire che "Dune – Parte 2" e "DUNE Parte 2" sono lo stesso film. Da Wikidata arrivano locandina, regista e trama. "Cosa è cambiato" tiene i dati vecchi finché non arrivano quelli nuovi.

In produzione gira da solo ogni notte alle 3. **75 test automatici** controllano che ogni modulo funzioni.

Backend: un'API che serve i dati all'app

Il backend è organizzato a livelli: ogni livello ha una responsabilità sola e chiama solo quello sotto.



Gli 11 endpoint (visibili su /docs)

Film — di oggi, della settimana, ricerca, dettaglio

Cinema — lista, singolo, spettacoli del cinema

Spettacoli — per una data

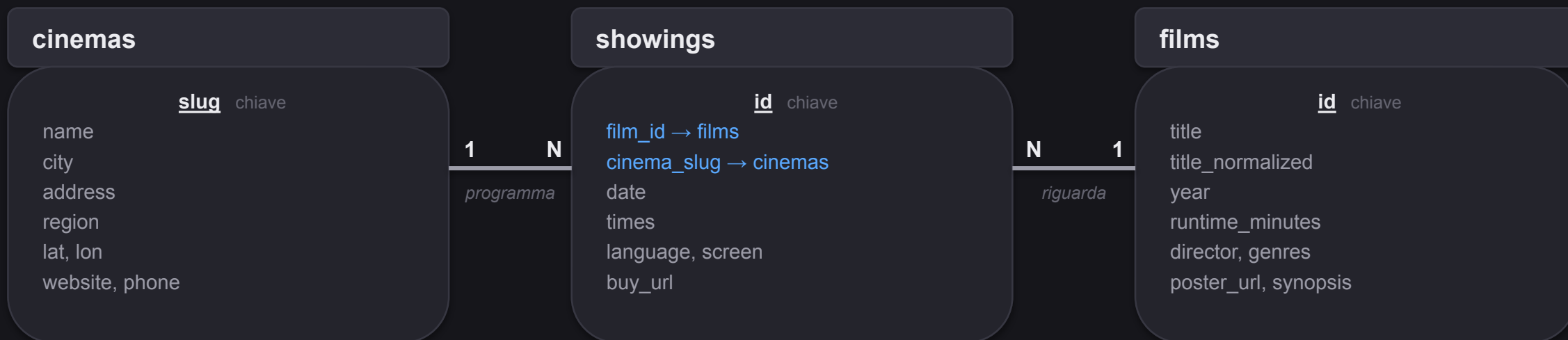
Amministrazione — ricarica dati e conteggi (con token)

Stato — health check

Sono in sola lettura per l'app; solo i due di amministrazione sono protetti.

Tecnologie: **FastAPI**, **SQLAlchemy**, **SQLite**, **Pydantic**. I dati si caricano dai file dello scraper in modo ripetibile (ricaricarli non crea doppi). **26 test** coprono database ed endpoint.

Il database: tre tabelle, due relazioni



Ogni spettacolo (showings) lega un film a un cinema in una certa data, con i suoi orari. Un cinema ha molti spettacoli, un film ha molti spettacoli: showings sta in mezzo alle due tabelle.

Due accorgimenti importanti: **la coppia film–cinema–data è unica**, così ricaricare i dati non crea doppioni; **cancellando un film o un cinema spariscono anche i suoi spettacoli**.

L'app: un solo codice per web e telefono

Con cosa è costruita

React Native — il framework dell'app

Expo — strumenti e anteprima sul telefono

React Navigation — le schede in basso e le pagine di dettaglio

react-native-web — lo stesso codice gira anche nel browser

Leaflet — la mappa dei cinema

AsyncStorage — i preferiti, salvati sul telefono

Come ci si muove nell'app

All'avvio parte una breve animazione, poi si arriva alle tre schede in basso:

Films — la home con il cartellone

Cerca — la ricerca per titolo

Località — la mappa dei cinema

Da Films e da Cerca si apre il dettaglio del film restando dentro la stessa scheda, con la barra sempre visibile in basso.

Le schermate e qualche scelta tecnica

Films (home)

carosello in alto, barra dei sette giorni e griglia delle locandine; si può filtrare per cinema e aggiornare tirando giù

Cerca

cerca per titolo mentre scrivi; aspetta un attimo prima di interrogare il backend e ignora le risposte più vecchie

Località

mappa con i tre cinema e i loro indirizzi; toccandone uno si apre in Google Maps

Dettaglio film

locandina, trama, trailer e gli orari raggruppati per cinema, aperti già sulla data giusta

Un dettaglio di cura: le locandine dei cinema sono a bassa risoluzione, così le mostriamo piccole e nitide su uno sfondo sfocato, invece che grandi e sgranate.

Gli schemi di progettazione usati

Schema (pattern)	Dove	A cosa serve
Strategy	i moduli dello scraper	stessa interfaccia, una tecnica diversa per ogni cinema
Repository	backend, accesso ai dati	tiene le query in un posto solo, staccate dal resto
Service	backend, logica	separa le regole di business dal web e dal database
DTO	gli oggetti Pydantic	definiscono la forma esatta dei dati verso l'app
Dependency Injection	il database in FastAPI	nei test si sostituisce con un database in memoria
Factory	creazione dell'app	un unico punto che costruisce e configura tutto

Come teniamo il progetto in salute

101

test automatici

75

sullo scraper

26

sul backend

Controllo automatico

A ogni modifica caricata, i test girano da soli su GitHub. Se qualcosa si rompe, ce ne accorgiamo subito.

Un problema vero, trovato e risolto

The Space propone lo stesso film in più sale. La regola che tiene unici i dati ne conservava una sola: "Odissea" mostrava un orario invece di dodici.

Ora, prima di salvare, uniamo gli orari di tutte le sale. Trovato il motivo, sistemato, e aggiunto un test perché non ricapiti. Da uno a dodici orari.

I rischi principali e come li gestiamo

Rischio	Gravità
Un sito cambia e lo scraper si rompe	Alta
Un sito è troppo dinamico da leggere	Alta
Le regole del sito vietano la raccolta	Media
Si aggiungono funzioni non previste	Media
Wikidata non ha un film di nicchia	Bassa

Come teniamo a bada il primo

- un modulo per ogni cinema: si rompe solo quello
- gli errori restano isolati, gli altri vanno avanti
- teniamo i dati vecchi finché non arrivano i nuovi
- i test segnalano subito se qualcosa cambia

Raccolta dati corretta e trasparente Rispettiamo le regole dei siti, una richiesta al secondo, di notte, con un contatto reale nell'intestazione. E rimandiamo sempre al sito del cinema per l'acquisto: **portiamo traffico, non lo sottraiamo.**

Tre scelte che difendiamo

Usare SQLite anche in produzione, non un database più grande

Tre cinema e poche centinaia di spettacoli, in sola lettura: un database server aggiungerebbe lavoro senza vantaggi. E se un domani servisse, cambia solo una riga di configurazione, non il codice.

Prendere i dati dei film da Wikidata, non da servizi a pagamento

Nessuna chiave da gestire, nessun limite commerciale, licenza aperta; teniamo una copia locale per non ripetere le richieste.

Un modulo di raccolta separato per ogni cinema

Se un sito si rompe non trascina gli altri, e possiamo adattarci a fonti diverse, anche a quelle non documentate come UCI.

Vediamola dal vivo

L'app, gli orari veri raccolti stamattina, tutta la catena dall'inizio alla fine.

In due parole

Quello che c'è oggi

- la catena completa, dai siti fino all'app, su dati veri
- un'unica app che gira su web e su telefono
- 101 test automatici e controllo a ogni modifica

Quello che verrà dopo

- l'avviso quando i dati di un cinema sono vecchi
- la pubblicazione online di backend e app
- la ricerca per titolo, che però è già pronta

Cosa diciamo con onestà

Quando l'anno di un film manca, i doppioni li togliamo via codice e non col database. L'API di UCI non è ufficiale e può cambiare. SQLite scrive un po' alla volta: per noi va bene, perché scrive solo di notte.

Grazie. — Team RepCode